



Universidad Nacional Autónoma de México

Facultad de Ciencias

Proyecto Final

Definición Dirigida por Sintaxis para un lenguaje tipo C

Nombre de los integrantes del equipo:

- Brenda Jiménez Ruiz
- Irany Solano Marcial
- José Alejandro Rojo Quezada
- Yessica Vianney Montes de Oca Aguilar

Nombre de la Profesora: Ing. Adara Mercado Martínez

Nombre de los Ayudantes:

- Luis Mario Escobar Rosales
- Carlos Gerardo Acosta Hernández

Grupo: 7013



Lista de tokens del analizador léxico

Token	Lexema(s)	Categoría	Valor semántico
Tipos base			
INT	"int"	tipo base	—
FLOAT	"float"	tipo base	—
CHAR	"char"	tipo base	—
BOOL	"bool"	tipo base	—
VOID	"void"	tipo base	—
Palabras reservadas			
IF	"if"	reservada	—
ELSE	"else"	reservada	—
WHILE	"while"	reservada	—
FOR	"for"	reservada	—
BREAK	"break"	reservada	—
RETURN	"return"	reservada	—
DEF	"def"	reservada	—
STRUCT	"struct"	reservada	—
Literales (con valor semántico)			
ID	[a-zA-Z_] [a-zA-Z0-9_]*	literal	string — el nombre
NUMERO	[0-9]+	literal	string — valor entero
FLOTANTE	[0-9]+. [0-9]+([eE] [+]?[0-9]+)?	literal	string — valor float
BOOLEANO	"true", "false"	literal	string — "true"/"false"
STRING	"..."	literal	string — contenido
CHARACTER	'.'	literal	string — el carácter
Operadores aritméticos			
MAS	+	operador	—
MENOS	-	operador	—
MUL	*	operador	—
DIV	/	operador	—
MOD	%	operador	—
INC	++	operador	—
DEC	-	operador	—
Operadores relacionales y lógicos			

Continúa en la siguiente página...



Token	Lexema(s)	Categoría	Valor semántico
EQUAL	==	relacional	—
DISTINCT	!=	relacional	—
LT	<	relacional	—
GT	>	relacional	—
LE	<=	relacional	—
GE	>=	relacional	—
AND	&&	lógico	—
OR		lógico	—
NOT	!	lógico	—
Asignación			
ASIG	=	asignación	—
Delimitadores y puntuación			
LPAR	(	delimitador	—
RPAR	)	delimitador	—
LKEY	{	delimitador	—
RKEY	}	delimitador	—
LBRA	[	delimitador	—
RBRA	]	delimitador	—
PYC	;	puntuación	—
COMA	,	puntuación	—
PUNTO	.	puntuación	—



1 Programa Fuente Analizado

Listing 1: Programa de entrada (pág. 17 DDS_2026v1.pdf)

```
1 struct X {
2     int a;
3     int b;
4     float c;
5 } a, b;
6
7 float x, y;
8 int m, n;
9 int c;
10 int i, j;
11 float[5][7][3] w;
12 int z;
13 int w1;
14
15 def int suma(int x, int y) {
16     w1 = 5;
17     z = x + y + w1;
18     return z;
19 }
20
21 def int main() {
22     a.a = 8;
23     a.b = 2;
24     a.c = 1.5;
25     m = 7;
26     n = 15;
27     c = suma(m, n);
28     for(i=0; i<5; i=i+1) {
29         for(j=0; j<7; j=j+1) {
30             for(k=0; k<3; k=k+1)
31                 a[i][j][k] = 0;
32         }
33     }
34     for(i=0; i<5; i=i+1) {
35         for(j=0; j<7; j=j+1) {
36             for(k=0; k<3; k=k+1)
37                 b[i][j][k] = b[i][j][k] + 3;
38         }
39     }
40     return 0;
41 }
```

Mediante la definición dirigida por sintaxis, es decir; siguiendo las reglas gramaticales junto con las reglas semánticas genere los siguientes datos:

A continuación mostraremos el árbol sintáctico por partes para una mejor visualización pero se puede observar completo en nuestro repositorio de GitHub y en la pagina de draw.io.



Compiladores

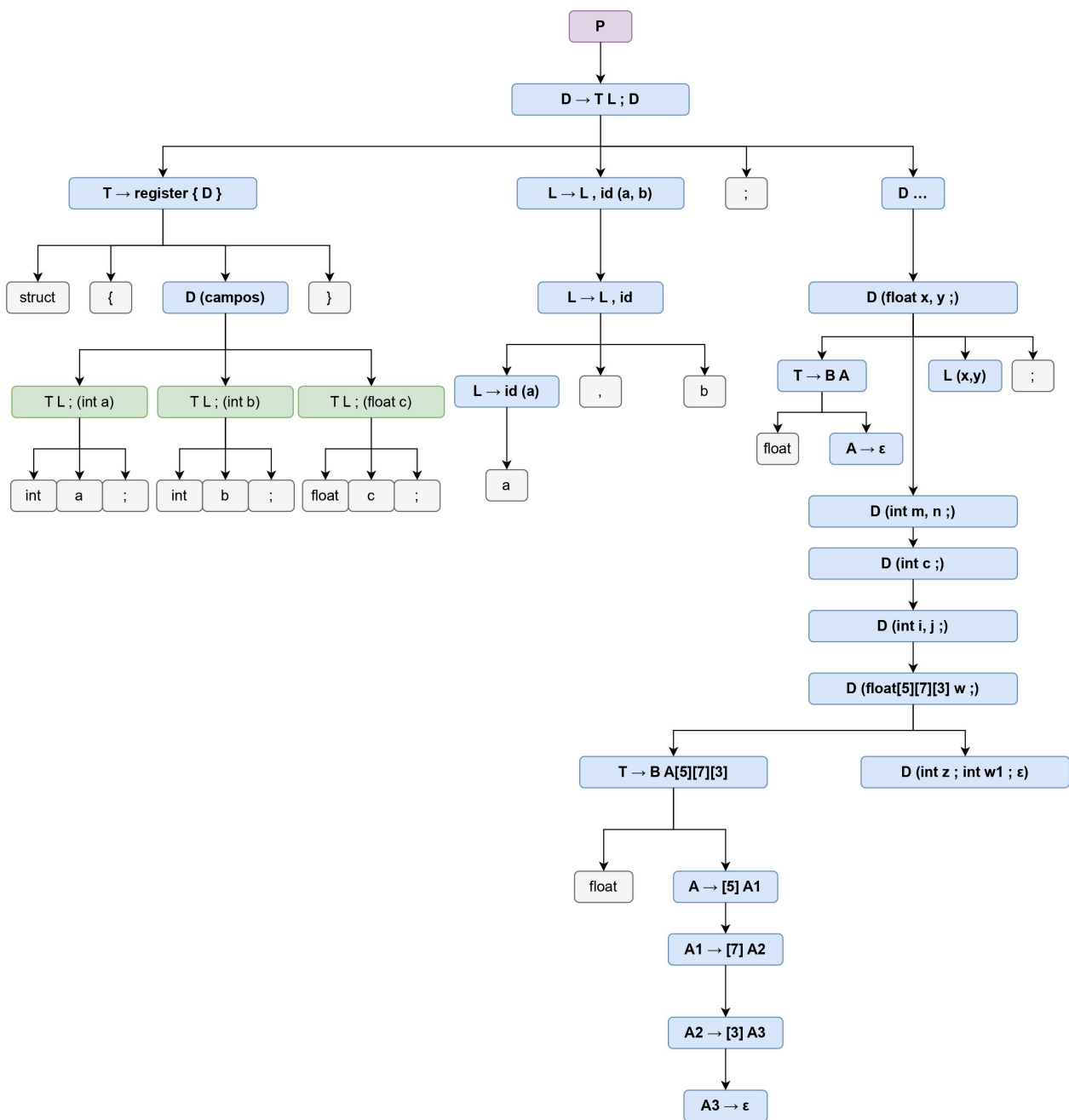
El árbol sintáctico completo se puede observar en GitHub en el siguiente enlace: [Árbol Sintáctico en GitHub](#)

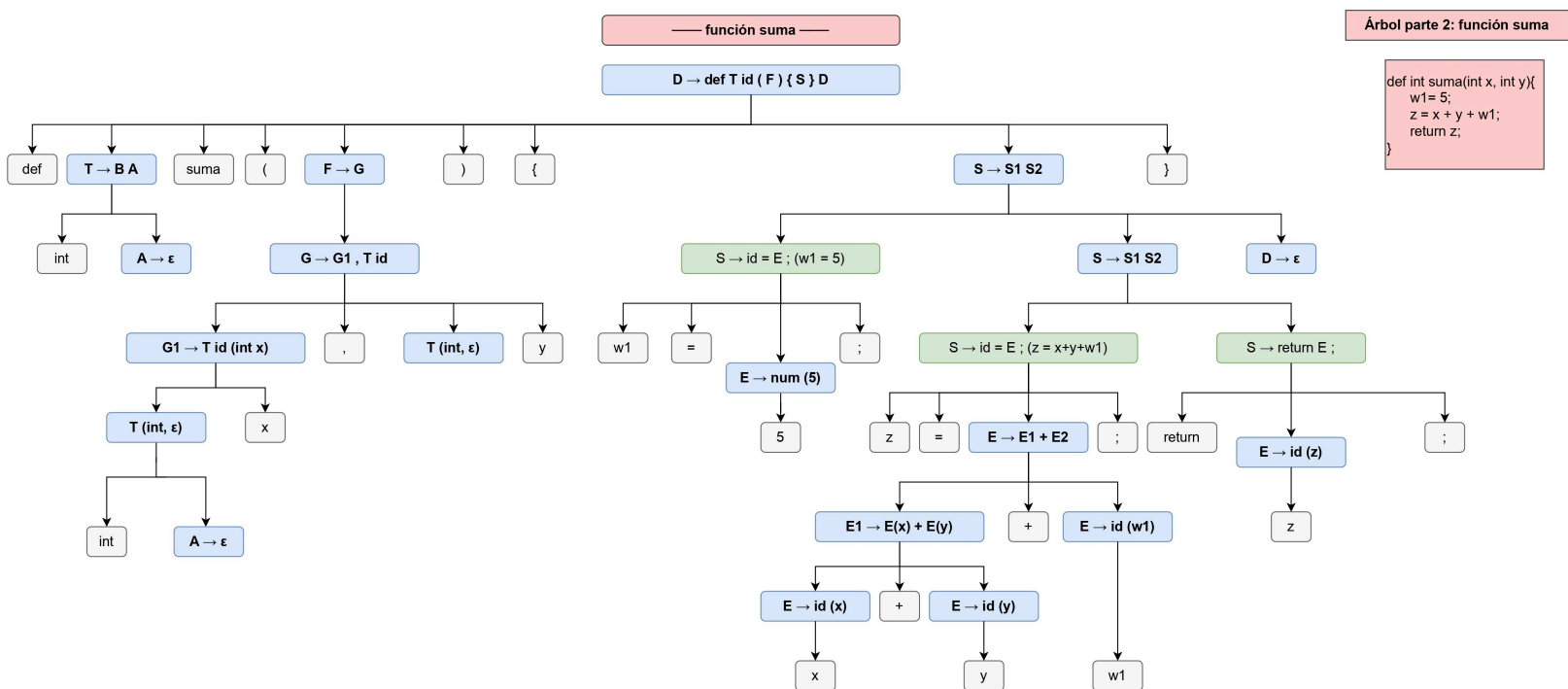
El árbol sintáctico completo se puede observar en draw.io en el siguiente enlace: [Árbol Sintáctico en draw.io](#)

Árbol parte 1: declaraciones globales.

```
struct X {
  int a;
  int b;
  float c;
}a, b;

float x, y;
int m, n;
int c;
int i, j;
float[5][7][3] w;
int z;
int w1;
```

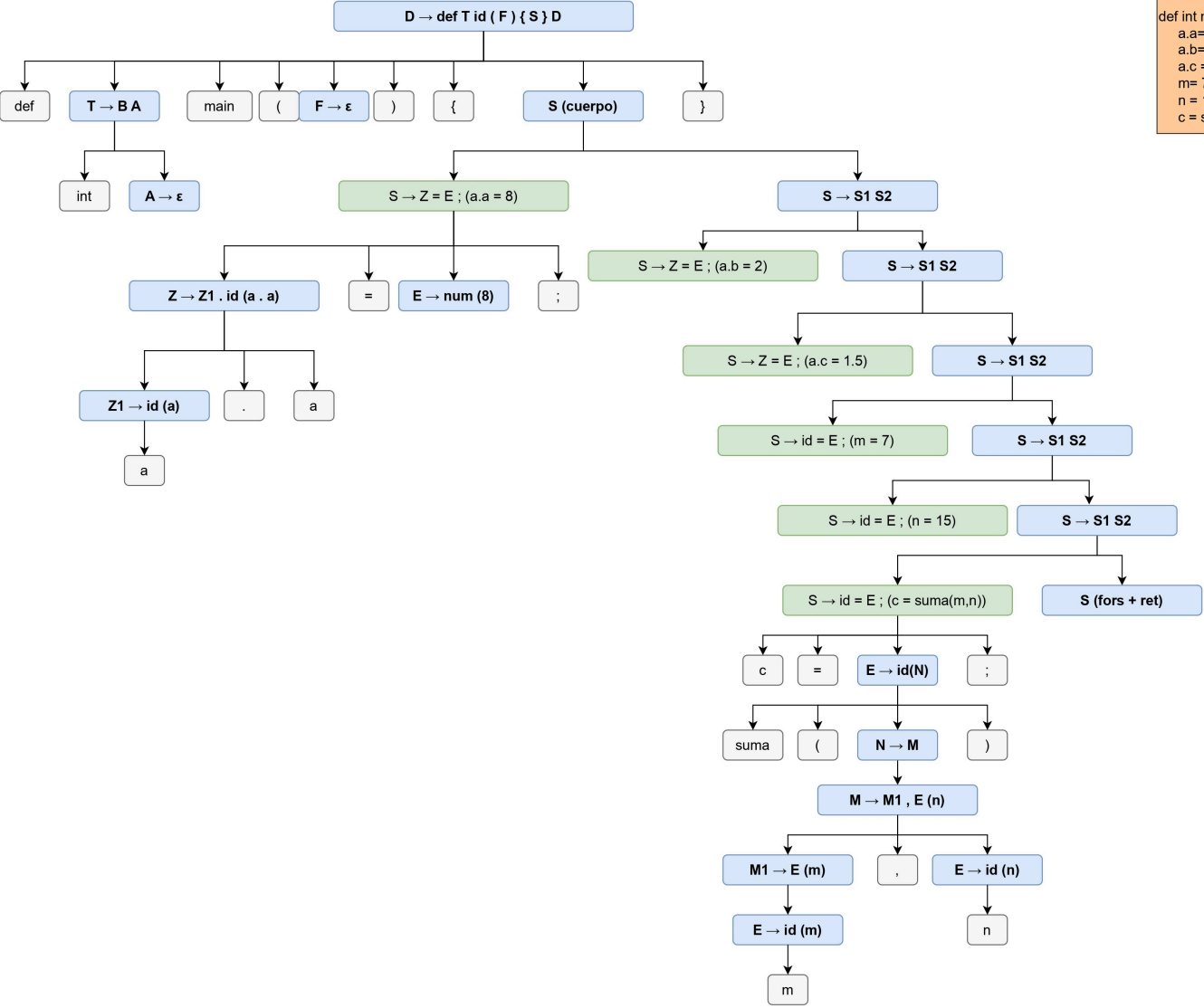




función main

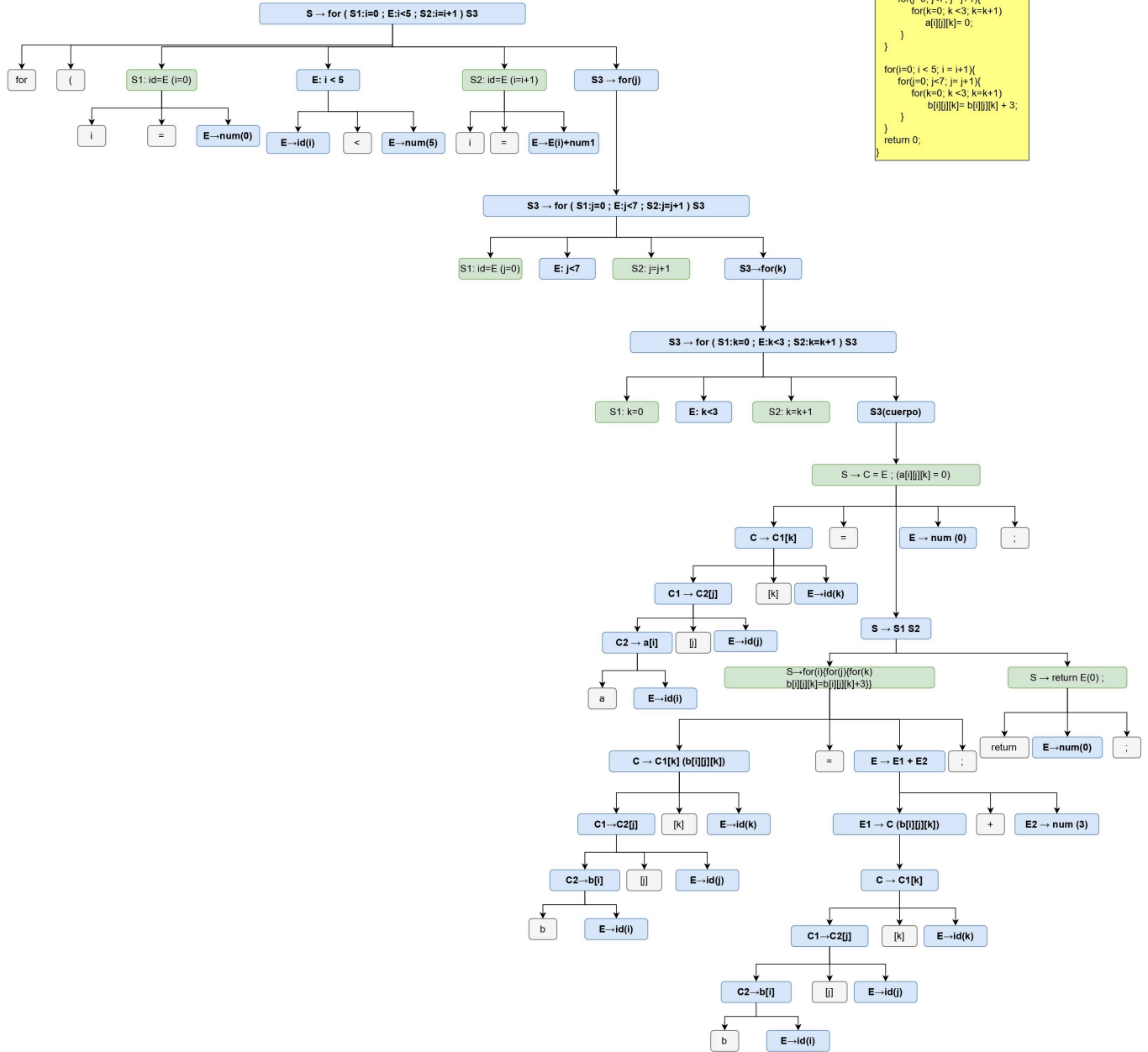
Árbol parte 3a: cabecera de main y sentencias iniciales

```
def int main(){
  a.a= 8;
  a.b= 2;
  a.c = 1.5;
  m= 7;
  n = 15;
  c = suma(m, n);
}
```



— for anidados × 2 bloques —

Árbol parte 3b: los dos bloques de for triplemente anidados de main



```

for(i=0; i < 5; i = i+1){
  for(j=0; j<7; j = j+1){
    for(k=0; k < 3; k=k+1)
      a[i][j][k]= 0;
  }
}

for(i=0; i < 5; i = i+1){
  for(j=0; j<7; j = j+1){
    for(k=0; k < 3; k=k+1)
      b[i][j][k]= b[i][j][k] + 3;
  }
}
return 0;

```



Compiladores

1. El grafo de dependencias
2. La pila de tablas de símbolos.
3. La tabla de tipos.
4. El código intermedio